

USC Write-Ability — Relay PoC

USC Write-Ability — Relay PoC: design brief

Field	Value
Audience	Protocol, relay operators, smart contracts, integrations
Scope	Initial proof of concept for relay clients : observe votes, assemble calldata, deliver to inbox on destination chains
Paired specs	confluence-attestor-write-ability-poc.md (votes); confluence-quoter-production-spec.md (fees — §2.3 interconnection)
Related research	01-architecture-overview.md, 02-message-protocol.md, 04-inbox-contract.md, 12-signature-validation-implementation.md, requirements/02-relayers-requirements.md

1. Executive summary

Relayer clients are off-chain services that:

1. Learn which messages exist on **Creditcoin L1** (via `MessagePublished` on the **outbox** for a given destination / `chain_key`).
2. **Snoop** the **Message Vote Gossip (MVG)** P2P topic (same as attesters: `{chain_key}/message-votes/v1`) and/or use HTTP helpers if exposed.

3. **Aggregate** distinct valid attester signatures until the **same threshold** the destination **inbox** enforces (typically **2/3+1** of the configured attester set).
4. Submit `deliverMessage(messageId, emitterAddress, payload, votes)` on the **destination chain's inbox**, paying **native gas** on that chain.

Relayers are **not** part of the security model: they cannot forge votes; they only **transport** evidence produced by attesters. They **must not** run inside the attester process (CTO constraint: **attester $\neq$ relayer**).

2. Role and trust boundaries

Party	Responsibility
Attesters	Finalize <code>MessagePublished</code> , sign <code>messageHash</code> , gossip votes.
Relayers	Collect votes, call inbox on client chain, manage gas and retries.
Inbox + <code>IVoteValidator</code>	Recompute <code>messageHash</code> , verify <code>votes</code> , execute <code>receiveMessage</code> on dApp.

Trust: Users rely on **attester quorum + inbox math**, not on relayer honesty for *validity*. A malicious relayer can **censor** (not deliver) or **waste gas** on failing txs; it cannot pass `validateVotes` without a real threshold of valid signatures over the correct `messageHash` .

2.1 Interconnection with the quoter (economic vs delivery)

They are not the same program and they do not call each other. The link is **economic** and **on-chain**, not a wire protocol between quoter and relayer client.

Piece	Role in the interconnection
Quoter (off-chain)	dApp (or backend) calls <code>POST /v1/quote</code> → receives <code>signedQuote</code> bytes pricing <code>relayPrice</code> + optional <code>acknowledgmentPrice</code> .

RelayerContract (on client chain EVM)	dApp calls <code>validateAndCollectFee(signedQuote)</code> before publishing on Creditcoin; contract checks whitelisted quoter signature, expiry, token , then pulls payment to <code>payeeAddress</code> (relayer pool).
Relayer client (off-chain)	Never consumes the quoter API for delivery. It pays destination gas from its own hot wallet, watches P2P votes , and calls <code>Inbox.deliverMessage</code> . It is incentivized because the pool (funded by quotes) later compensates relayers per your pool rules.

End-user flow (typical):

1. dApp → **quoter** → gets signed quote.
2. dApp → RelayerContract on **destination** chain → fee taken, funds go to **pool**.
3. dApp → **outbox** on **Creditcoin** → `MessagePublished`.
4. Attesters → **P2P votes**.
5. **Relayer client** → `deliverMessage` on **destination** inbox (gas from relayer wallet).

So: **quoter + relayer contract = who paid and how much; relayer client + inbox = message actually moved**. Details: `confluence-quoter-production-spec.md` §2.3, `07-quotation-system.md`.

Is RelayerContract “tied to” relayers? Yes in the **network** sense: research describes **one RelayerContract** per client chain as that relayer network’s on-chain face — it receives payments and routes them to `payeeAddress` (typically a **relayer pool**). Off-chain **relayer clients** are the same *business* network but a **different component**: they execute delivery and are compensated **via that pool / incentive rules**, not by taking user funds directly.

Why don’t relayer clients collect fees? By design (`07-quotation-system.md`): **fee collection is on-chain** when the **dApp** calls `validateAndCollectFee(signedQuote)`. That gives (1) a **fixed contract address** and **verifiable** quoter signature + expiry, (2) no need to trust

whichever relay EOA might deliver later, (3) no race over “which relay gets paid before doing work.” Relay clients stay focused on **gas + delivery**; **economic settlement** stays in `RelayerContract` + pool.

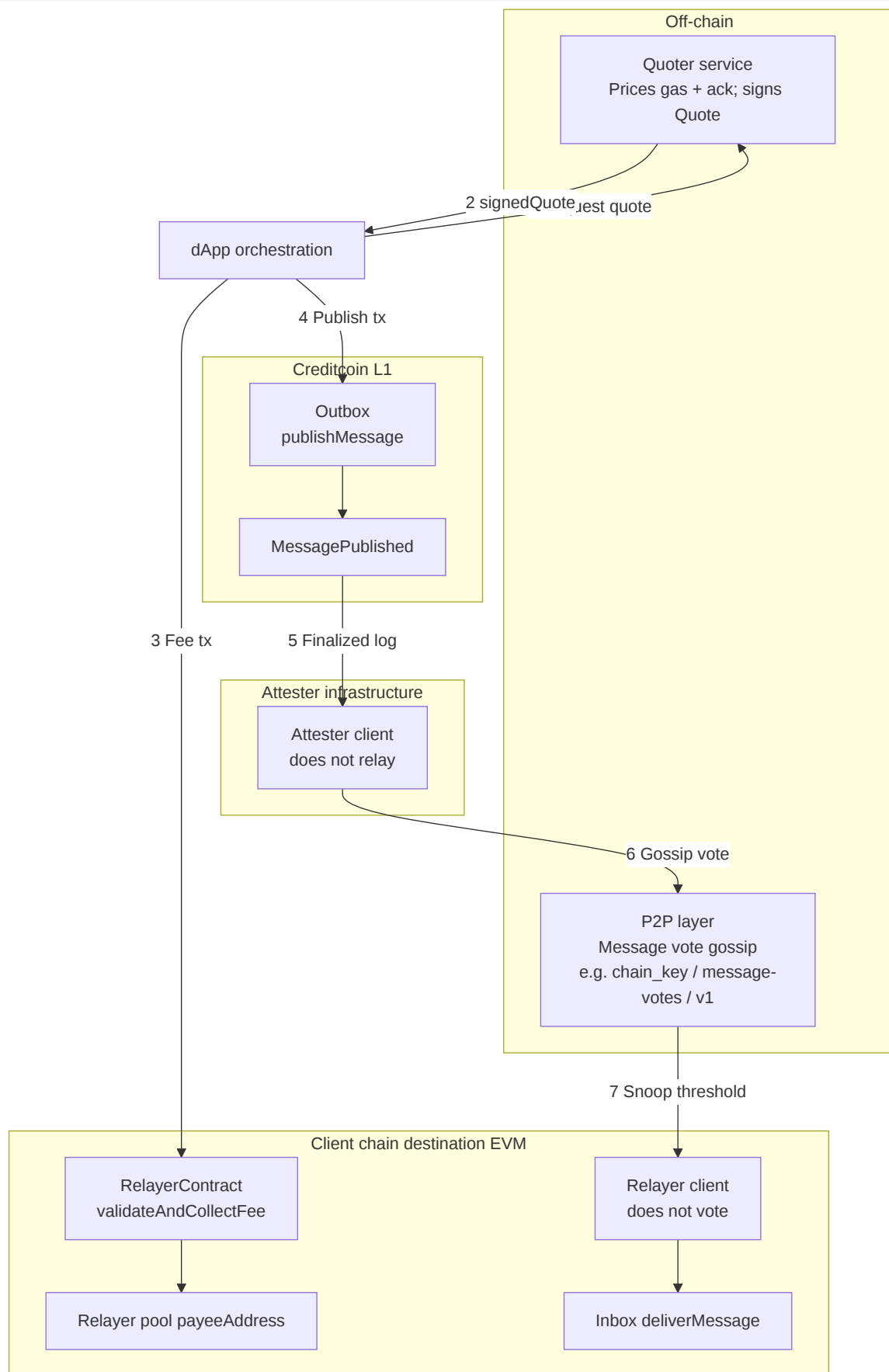
2.2 Research-aligned flow diagram

Placement (per 02-relayers-requirements.md , 07-quotation-system.md , 02-message-protocol.md): `RelayerContract` and `Inbox` live on the **client (destination) chain**; `Outbox` lives on **Creditcoin L1**. The quoter is **off-chain**. **Attester client ≠ relay client** (01-attesters-requirements.md). The relay contract **never** calls the outbox.

Source file (editable): `usc-write-ability-research/documents/mermaids/08-quoter-relayer-attester-message-flow.mermaid`

▼ mermaid diagram

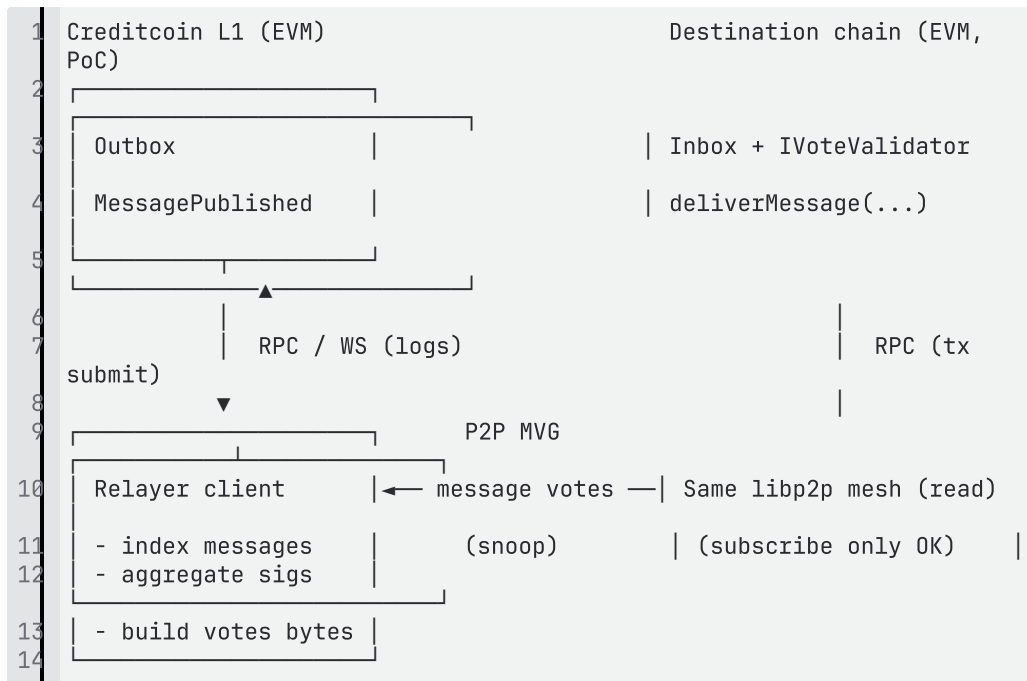
```
1 flowchart TB
2     subgraph OFF["Off-chain"]
3         QUOTER["Quoter service<br/>Prices gas + ack; signs Quote"]
4         P2P["P2P layer<br/>Message vote gossip<br/>e.g. chain_key /
5 message-votes / v1"]
6     end
7
8     subgraph CC["Creditcoin L1"]
9         OUTBOX["Outbox<br/>publishMessage"]
10        EVT["MessagePublished"]
11        OUTBOX --> EVT
12    end
13
14    subgraph DEST["Client chain destination EVM"]
15        RELCON["RelayerContract<br/>validateAndCollectFee"]
16        POOL["Relayer pool payeeAddress"]
17        INBOX["Inbox deliverMessage"]
18        RELCLI["Relayer client<br/>does not vote"]
19        RELCON --> POOL
20        RELCLI --> INBOX
21    end
22
23    subgraph ATT["Attester infrastructure"]
24        ATCLI["Attester client<br/>does not relay"]
25    end
26
27    DAPP["dApp orchestration"]
28
29    DAPP -->|"1 Request quote"| QUOTER
30    QUOTER -->|"2 signedQuote"| DAPP
31    DAPP -->|"3 Fee tx"| RELCON
32    DAPP -->|"4 Publish tx"| OUTBOX
33    EVT -->|"5 Finalized log"| ATCLI
34    ATCLI -->|"6 Gossip vote"| P2P
35    P2P -->|"7 Snoop threshold"| RELCLI
```



Note: Steps 3 and 4 are **two separate on-chain transactions**, often on **different chains**; the diagram uses one **dApp orchestration** node to match the Solidity pattern in `07-quotation-`

system.md without implying a single-chain deployment.

3. Architecture (PoC)



Multi-chain: One relayer client **binary** may handle **many** (`creditcoin_chain_key`, `destination_rpc`, `inbox_address`) routes; each route has its own vote map and wallet (see `02-relayers-requirements.md`).

4. Configuration (per delivery route / PoC)

Parameter	Purpose
<code>chain_key</code>	USC / gossip topic scope (must match attesters for that destination).
Creditcoin EVM RPC	Read <code>MessagePublished</code> , <code>eth_chainId</code> , optional factory/outbox resolution.
Outbox address	Resolved via factory + <code>bytes32</code> chain key (same rules as attester spec) or overridden in PoC config.
Destination RPC URL	Submit <code>deliverMessage</code> txs.

Inbox contract address	On destination chain; must match deployment for <code>localChainKey</code> + <code>creditcoinChainId</code> .
<code>creditcoinChainId</code> / <code>localChainKey</code>	Must be consistent with inbox immutables when computing <code>messageHash</code> locally for verification.
P2P	Bootnodes, <code>chain_key</code> , subscribe to <code>{chain_key}/message-votes/v1</code> .
Signer / wallet	Funded EOA or service key on destination chain for gas.
Attester set metadata	Addresses (or mapping) that match <code>IVoteValidator</code> + off-chain signature verification — should match <code>active_attestor_set</code> / validator contract (see attester spec §6.3 signer alignment).

5. Message discovery (Creditcoin)

5.1 Primary source: `MessagePublished`

Relayers need `messageId`, `emitterAddress`, `payload`, and (implicitly) the same `destinationChainKey` and `creditcoinChainId` attestors used when signing.

Field	Source
<code>messageId</code> , <code>emitterAddress</code> , <code>payload</code>	Outbox event log
<code>destinationChainKey</code>	<code>Outbox.chainKey</code> (storage read on resolved outbox)
<code>creditcoinChainId</code>	<code>eth_chainId</code> from Creditcoin L1 RPC

5.2 Computing `messageHash` (verification before pay gas)

Relayer **must** compute:

```
messageHash = keccak256(abi.encode(messageId, emitterAddress,  
destinationChainKey, creditcoinChainId, payload))
```

Bit-exact `abi.encode` parity with Solidity is required so collected signatures are checked against the same hash attesters signed. Golden vectors: same approach as attester spec §6.2 (`confluence-attestor-write-ability-poc.md`).

5.3 Correlation ID

Internal state key: `(chain_key, messageHash)` (or `messageId` + outbox address if you prefer; `messageHash` is the commitment that votes bind to).

6. Vote collection (P2P snooping)

6.1 Protocol

- **Topic:** `{chain_key}/message-votes/v1` (must match attester MVG spec).
- **Envelope:** decode the same fields attesters publish (`message_hash`, `message_id`, `signature`, ... — see attester spec §6.5).

6.2 Validation (before counting)

Apply the same rules as attesters (**attester spec §5** summary):

Check	Action
Codec / version	Reject malformed.
Chain-first allowlist (recommended)	Only aggregate votes for <code>message_hash</code> that the relayer has indexed from a finalized <code>MessagePublished</code> (prevents junk maps).
Signature	<code>ecover(messageHash, sig)</code> matches expected format (65-byte ECDSA for reference <code>E0AValidator</code>).

Signer $\in$ attester allowlist	Use same list as on-chain validator (or subset).
Dedup	One vote per signer per <code>message_hash</code> .

6.3 Threshold

Use the **same formula** as the deployed `IVoteValidator` (e.g. **2/3+1** unique signers over total attesters **N**). See attester spec §6.6.

State: `HashMap<messageHash, BTreeMap<Signer, Signature65>>` (in-memory PoC is fine; optional persistence for restarts).

6.4 Building `votes` calldata

Matches attester spec §6.4:

```
1 | votes = abi.encode(bytes[] memory signatures);
```

Each signature is **65 bytes** `(r, s, v)` suitable for `ecrecover(messageHash, v, r, s)` in the reference EOA validator. **No duplicate signers** in the array (on-chain revert).

Ordering: Any order if unique; **sorting by signer address** is recommended for deterministic txs and easier debugging.

6.5 When to submit

- **First time:** when `unique_signers >= threshold`, submit `deliverMessage`.
- **Competition:** multiple relayers may try; **first successful** tx wins; others revert with **“Message already validated”** (inbox behavior per research) — idempotent client should treat as success.
- **Insufficient gas / execution failure:** inbox may leave message **pending**; relayer may call `retryPendingMessage(messageId)` (permissionless) per `04-inbox-contract.md`.

7. Delivery transaction (destination chain)

7.1 Calldata

Match `IInbox.deliverMessage`:

```
1 deliverMessage(
2     bytes32 messageId,
3     address emitterAddress,
4     bytes calldata payload,
```

```
5 bytes calldata votes
6 )
```

All values must be **byte-identical** to the attested message (payload from Creditcoin log).

7.2 Gas and operations

Concern	PoC guidance
Native balance	Relayer wallet on destination chain holds gas token (research: relayers need substantial native currency).
Nonce management	Serial tx submission per wallet; optional multiple wallets for throughput (out of PoC scope).
Simulation	<code>eth_call</code> / <code>simulateContract</code> before send to catch <code>validateVotes</code> revert.
Monitoring	Track <code>MessageDelivered</code> / <code>MessagePending</code> to confirm outcome.

7.3 Relay contract / quotes (optional for PoC Phase 1)

Production flow: dApp pays via **relay contract** on client chain before publishing on L1 (`02-relayers-requirements.md`). **PoC Phase 1** may skip fee integration and use a **dummy relay contract** or zero-fee dev path; **Phase 2+** wire `validateAndCollectFee` and quote validation.

8. Acknowledgments (out of PoC core, informational)

If `requiresAck = true`, **delivery infrastructure** later proves processing to Creditcoin using **native USC proving** (not message-vote P2P). Relayer clients may participate in **proof submission** per `10-delivery-acknowledgment.md` — treat as a **separate worker** from MVG vote snooping so concerns stay separable.

9. Abuse, flooding, and economics

Threat	Mitigation
--------	------------

P2P junk	Same validation as attesters; rate limits; optional chain-first allowlist (attester spec §5).
RAM exhaustion	Cap tracked messages; TTL/LRU on incomplete aggregates.
Censorship	Multiple relayer clients / networks compete; protocol does not enforce liveness of a specific relayer.
Gas grieving	Simulate before send; backoff on repeated <code>MessagePending</code> if dApp keeps reverting.

Relayers **do not** adjudicate message *content*; dApps enforce policy in `receiveMessage` .

10. Observability and SLAs (PoC)

Signal	Meaning
<code>messages_indexed</code>	Count of finalized <code>MessagePublished</code> seen.
<code>votes_per_message</code>	Histogram toward threshold.
<code>deliver_tx_submitted / succeeded / reverted</code>	Delivery health.
<code>time_to_threshold , time_to_deliver</code>	Latency SLOs for later tuning.

11. Relayer PoC — implementation checklist

11.1 Phase 0 — Skeleton

#	Task
L0	Repo / binary: <code>relayer-client</code> (or module in existing monorepo).

L1	Config loader per route (Creditcoin RPC, destination RPC, inbox, chain_key, P2P bootnodes).
----	--

11.2 Phase 1 — Creditcoin + hash

#	Task
L2	Resolve outbox (factory + <code>bytes32</code> or config override).
L3	Subscribe / poll <code>MessagePublished</code> with finality policy aligned to attesters.
L4	Implement <code>messageHash</code> builder; T1-style golden tests vs Solidity.

11.3 Phase 2 — P2P + aggregate

#	Task
L5	libp2p GossipSub subscriber on <code>{chain_key}/message-votes/v1</code> (read-only mesh participation acceptable).
L6	Vote validation + unique signer map + threshold logic (§6).
L7	Encode <code>votes</code> = <code>abi.encode(bytes[])</code> (§6.4).

11.4 Phase 3 — Deliver

#	Task
L8	Destination wallet + tx pipeline for <code>deliverMessage</code> .
L9	Handle already validated / pending outcomes; optional

	<code>retryPendingMessage</code> .
L10	Metrics + structured logs (§10).

11.5 Phase 4 — Product hooks (post-PoC)

#	Task
L11	Integrate relayer contract + quotation (07-quotation-system.md , 03-quotation-requirements.md).
L12	Profit / client registration (open questions in 02-relayers-requirements.md).

11.6 Testing

#	Task
T1	Hash + <code>votes</code> encoding: on-chain <code>validateVotes</code> passes in fork tests.
T2	E2E: dummy outbox publish → attesters gossip → relayer delivers to dummy inbox on testnet.
T3	Abuse: junk P2P does not blow memory or produce false submits (§9).
T4	Race: two relayers deliver; one succeeds, one gets expected revert / no double execution.

12. Open questions (from research + engineering)

#	Question
---	----------

1	Relayer registration: on-chain roster vs permissionless clients?
2	Profit distribution: on-chain vs off-chain; metrics (latency, success rate)?
3	Gas topping / IGP: if quoted gas insufficient, who refunds / tops up?
4	Multiple relayer networks: same outbox, different relayer contracts — routing rules?
5	Signer alignment: EVM addresses in <code>E0AValidator</code> vs Substrate attester accounts — operational mapping (attester spec §6.3).

13. References

Document	Path
Attester PoC (MVG, signing, abuse)	<code>usc-write-ability-research/documents/confluence-attestor-write-ability-poc.md</code>
Relayer requirements	<code>usc-write-ability-research/documents/requirements/02-relayers-requirements.md</code>
Inbox	<code>usc-write-ability-research/documents/04-inbox-contract.md</code>
Message protocol	<code>usc-write-ability-research/documents/02-message-protocol.md</code>

Vote validation	<code>usc-write-ability-research/documents/12-signature-validation-implementation.md</code>
Quotation system (fees vs outbox)	<code>usc-write-ability-research/documents/07-quotation-system.md</code>
End-to-end flow diagram (mermaid)	<code>usc-write-ability-research/documents/mermaids/08-quoter-relayer-attester-message-flow.mermaid</code>

End of document.